

# NCA Sponsored Training

Day 2 – Afternoon Session (Lab)

## Practical Introduction to Public Key Encryption

*Ahmed Didouh*

*Madinah*

*13/01/2026*



[rc3.kaust.edu.sa](http://rc3.kaust.edu.sa)





# Outline



## 1. How to encrypt files with OpenSSL

- *OpenSSL(RSA)*
- *OpenSSL(ECC)*

## 2. Digital Certificates and Signature

- *Examining details of a Certificate*
- *Root Certificate Authorities*

## 3. OpenSSL Demo

- *Complete communication protocol using asymmetric crypto*



# Asymmetric Cryptography



- The key objective of this lab is to provide a practical introduction to public key encryption, with a focus on RSA and Elliptic Curve methods. This includes the creation of key pairs and in the signing process.
- In the second part, we will explore Digital Certificates



# How to encrypt files with OpenSSL



## Get Started:

- Open a Linux terminal (Kali-linux Virtual Machine)
- Check if you have OpenSSL installed, and what version do you have:

**\$ openssl version**

- Generate a key pair with the following command

**\$ openssl genrsa -out private.pem 1024**



# How to encrypt files with OpenSSL



## Open the key:

- The file ***private.pem*** contains both the public and the private key.
- Open the file with the following command:

```
$ cat private.pem
```

- You can see the key's details with the following command

```
$ openssl rsa -in private.pem -noout -text
```



# How to encrypt files with OpenSSL



## Secure the key:

- Let's now secure the encrypted key with 3-DES:

```
$ openssl rsa -in private.pem -des3 -out key3des.pem
```

Why should you have a password on the usage of your private key?



# How to encrypt files with OpenSSL



## Export the public key:

- You can export the public key from the private one with the following command:

```
$ openssl rsa -in private.pem -out public.pem  
-outform PEM -pubout
```

- You can see the key's details with the following command

```
$ openssl rsa -in public.pem -pubin -text
```



# How to encrypt files with OpenSSL



## Encrypt and Decrypt a file:

- Create a file named “myfile.txt” and put a message into it.
- Encrypt your file with your public key:

```
$ openssl pkeyutl -encrypt -inkey public.pem -pubin  
-in myfile.txt -out top_secret.enc
```

- Try to open the encrypted file

```
$ cat top_secret.enc
```





# How to encrypt files with OpenSSL



## Encrypt and Decrypt a file:

- Decrypt your file with your private key:

```
$ openssl pkeyutl -decrypt -inkey private.pem -in  
top_secret.enc -out decrypted.txt
```

- Try to open the output file

```
$ cat decrypted.txt
```



# Digital Certificates and Signature



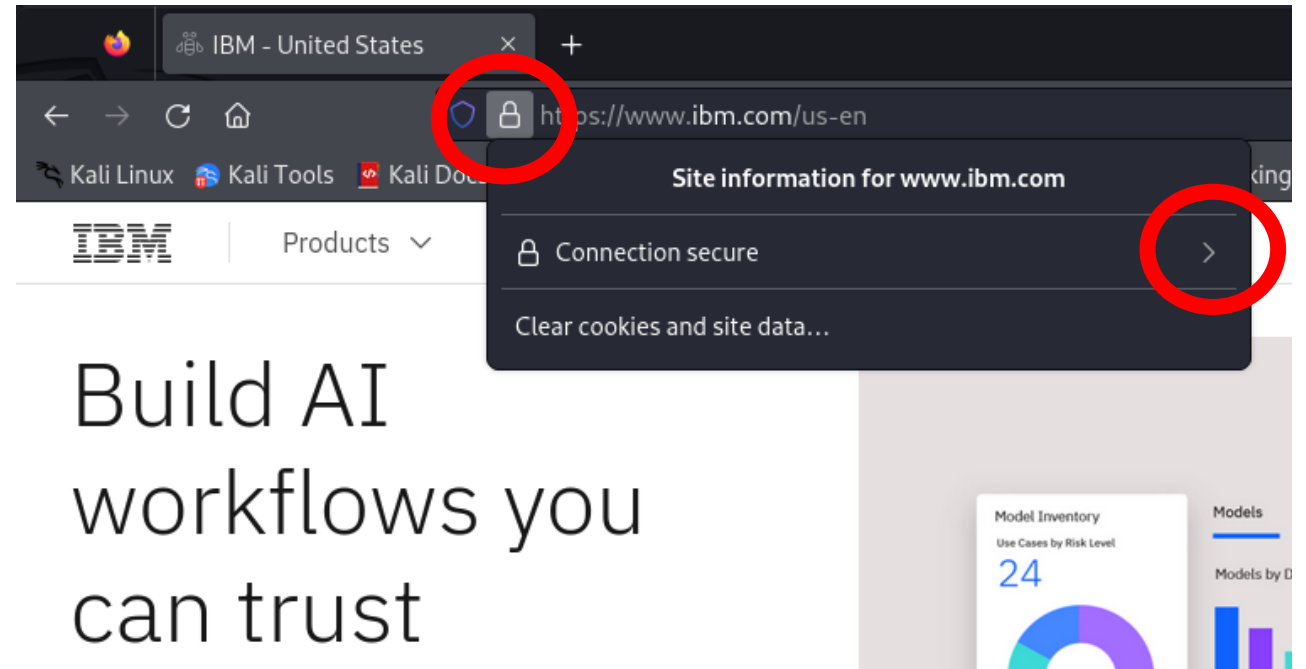
## Recap:

- A digital certificate “is essentially a **public key** accompanied by a **signature** of that key and associated information”
- (Aumasson 2018, Serious Cryptography).
- For TLS (**X.509**) certificates, this information includes the domain name, optionally alternative domain names that a cert is valid for, issue and expiration dates, and other details.



## Examining details of a Certificate

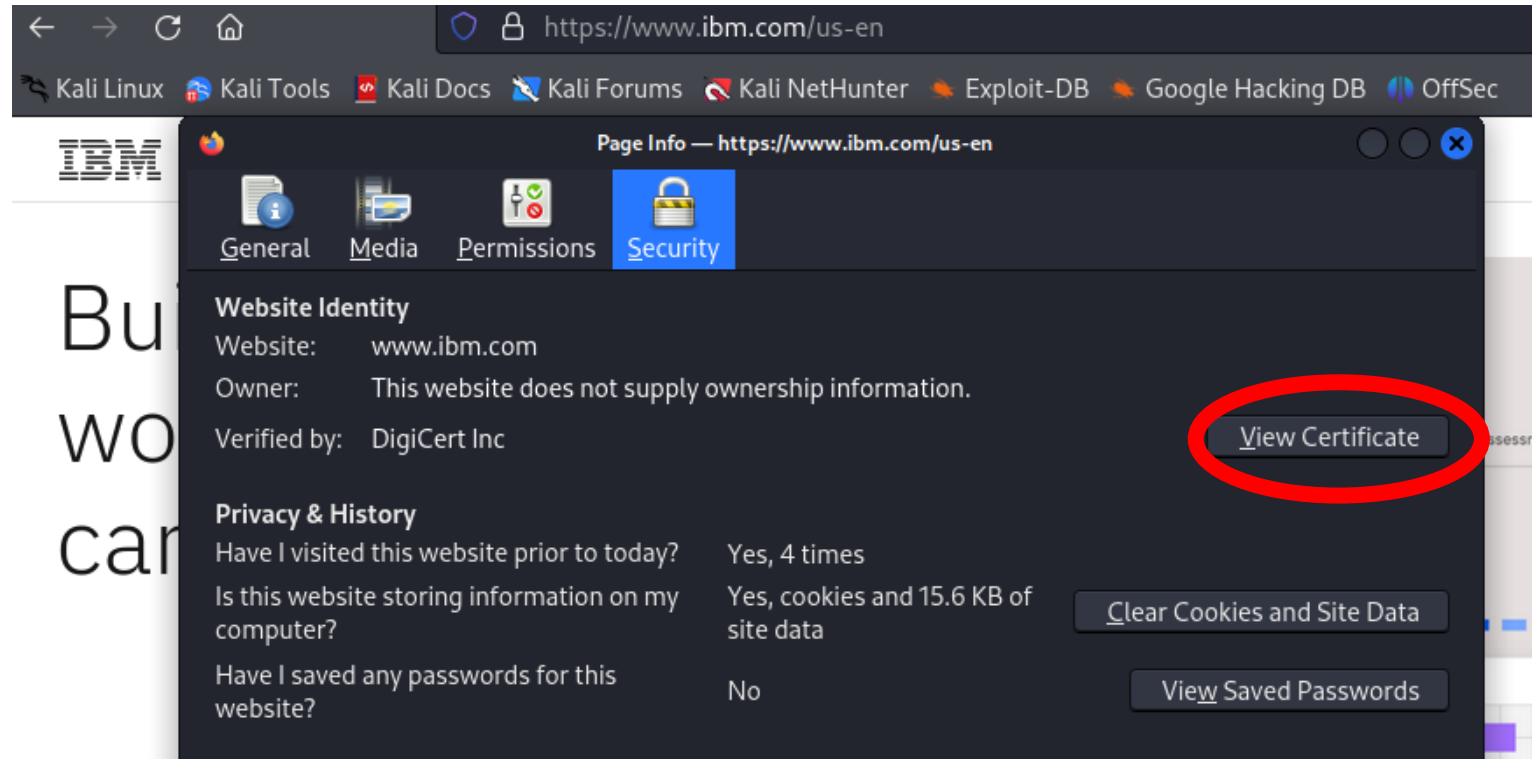
- Browse to **ibm.com** and inspect the certificate details (from Firefox, in your Kali-linux Virtual Machine)
- Click on the padlock icon in the URL bar.
- Click on the arrow to the right of the domain name





## Examining details of a Certificate

- Click on “More Information”
- Click “View Certificate”



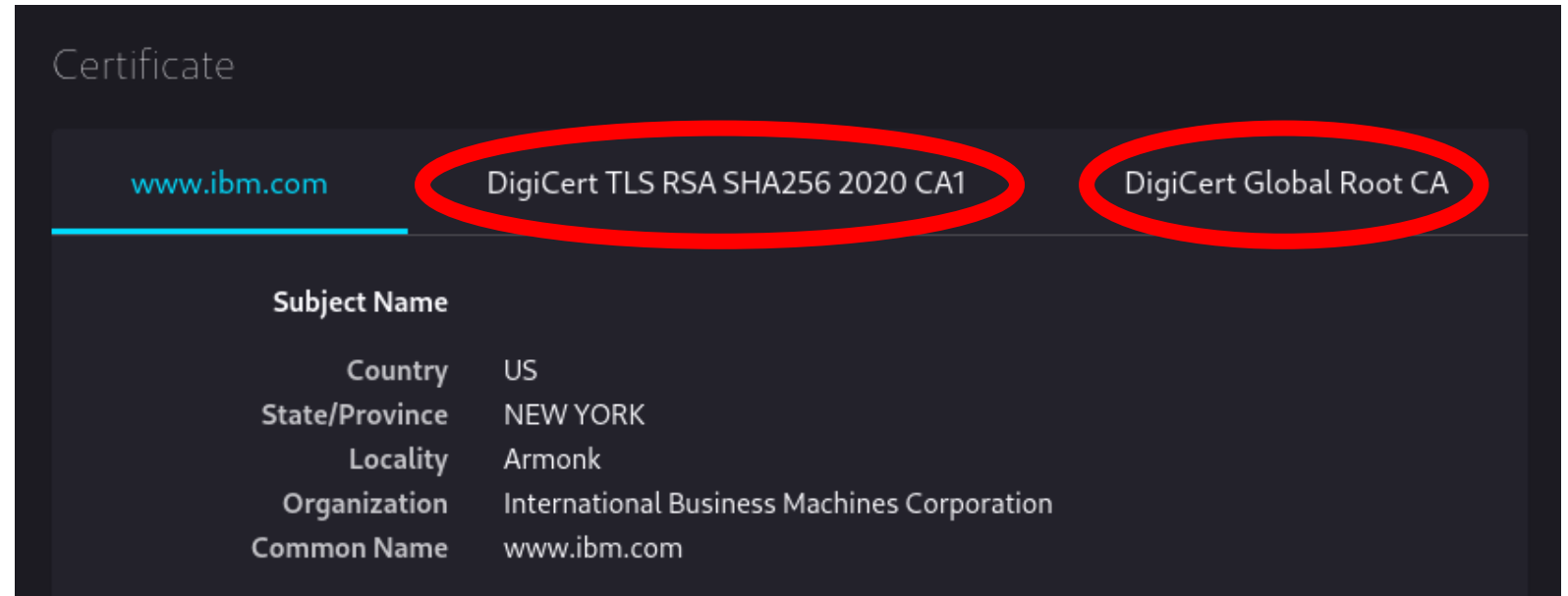


## Question 1:

What is the root certificate authority for ibm.com?

## Question 2:

What is the intermediate certificate authority?





## **Question 3:**

What are some of the other domains for which this certificate is valid?

## **Question 4:**

What algorithm does IBM's certificate use for their public key?

## **Question 5:**

What is the keysize of IBM's public key?

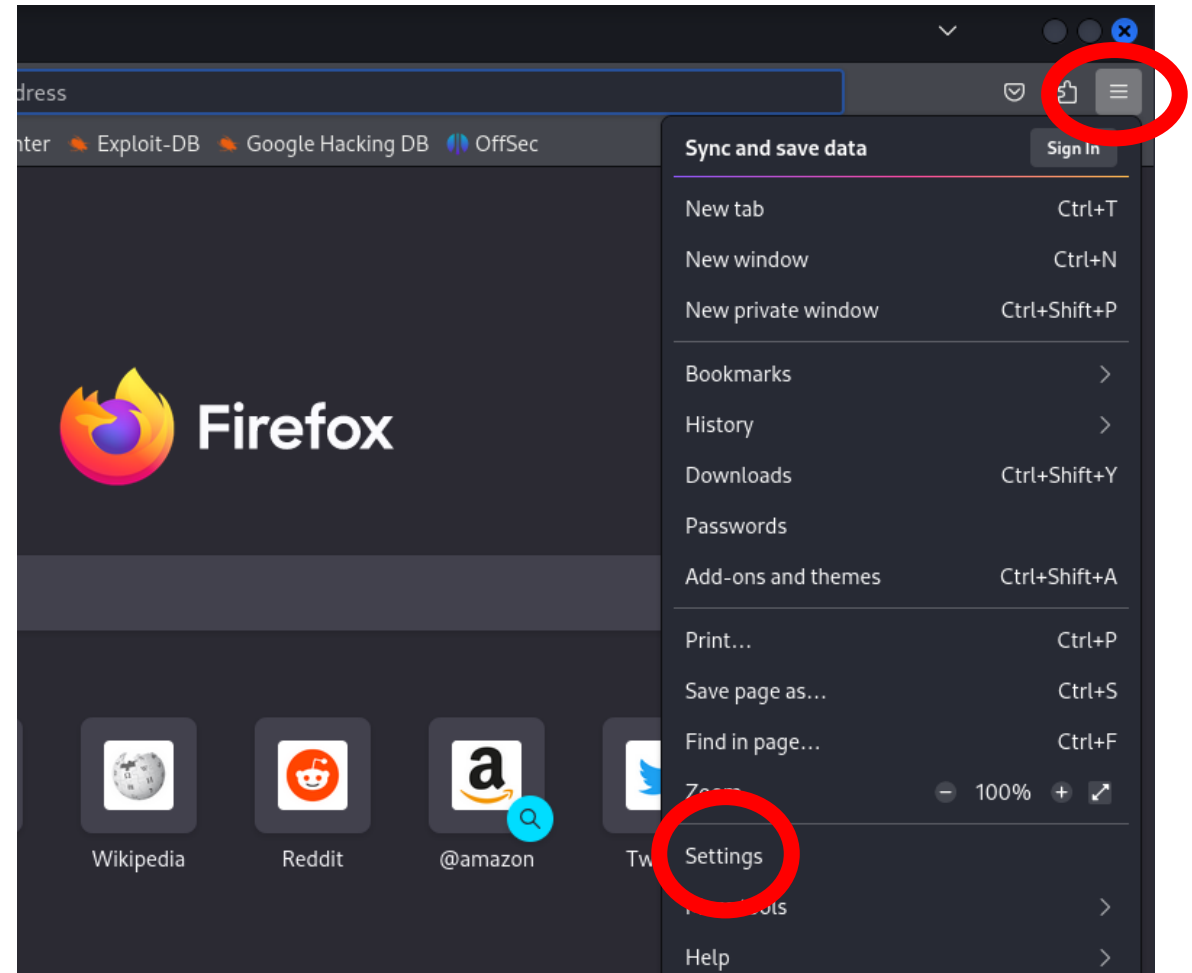


# Digital Certificates and Signature



## Root Certificate Authorities

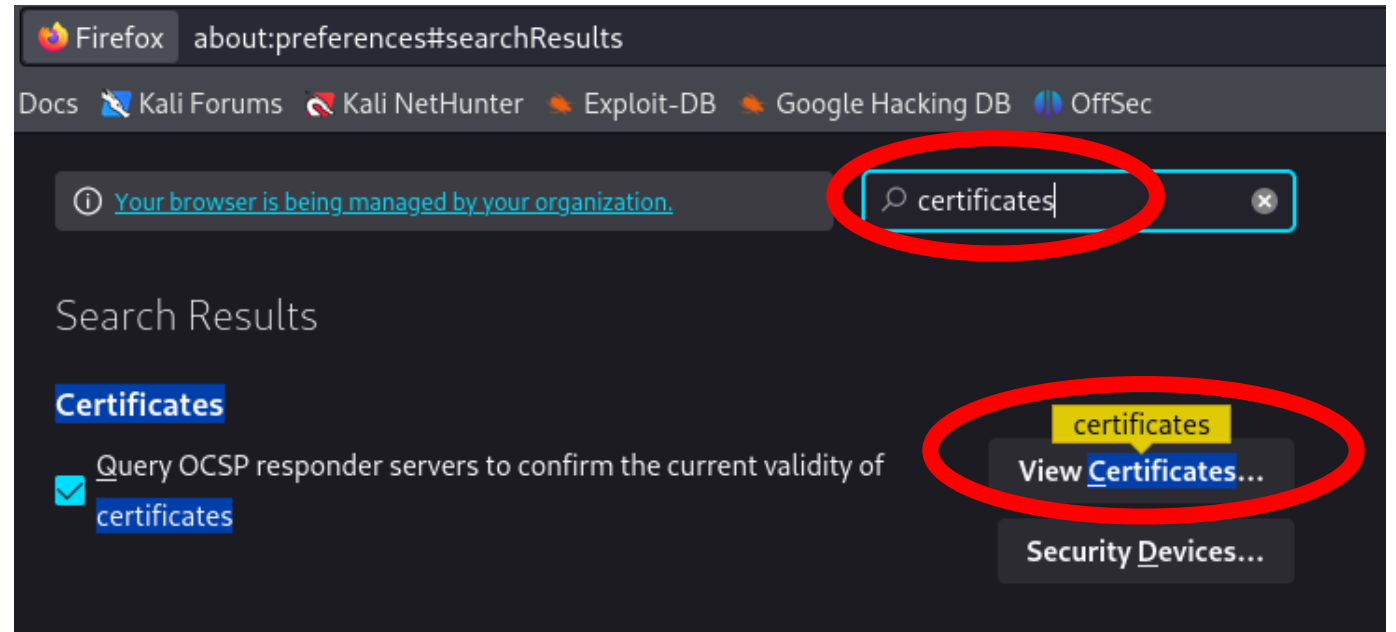
- In your Kali VM, open Firefox.
- Click on the “hamburger” icon in the top-right, and select “Settings”.





## Root Certificate Authorities

- Use the settings/preferences search bar to find the “View Certificates” button. Click it.

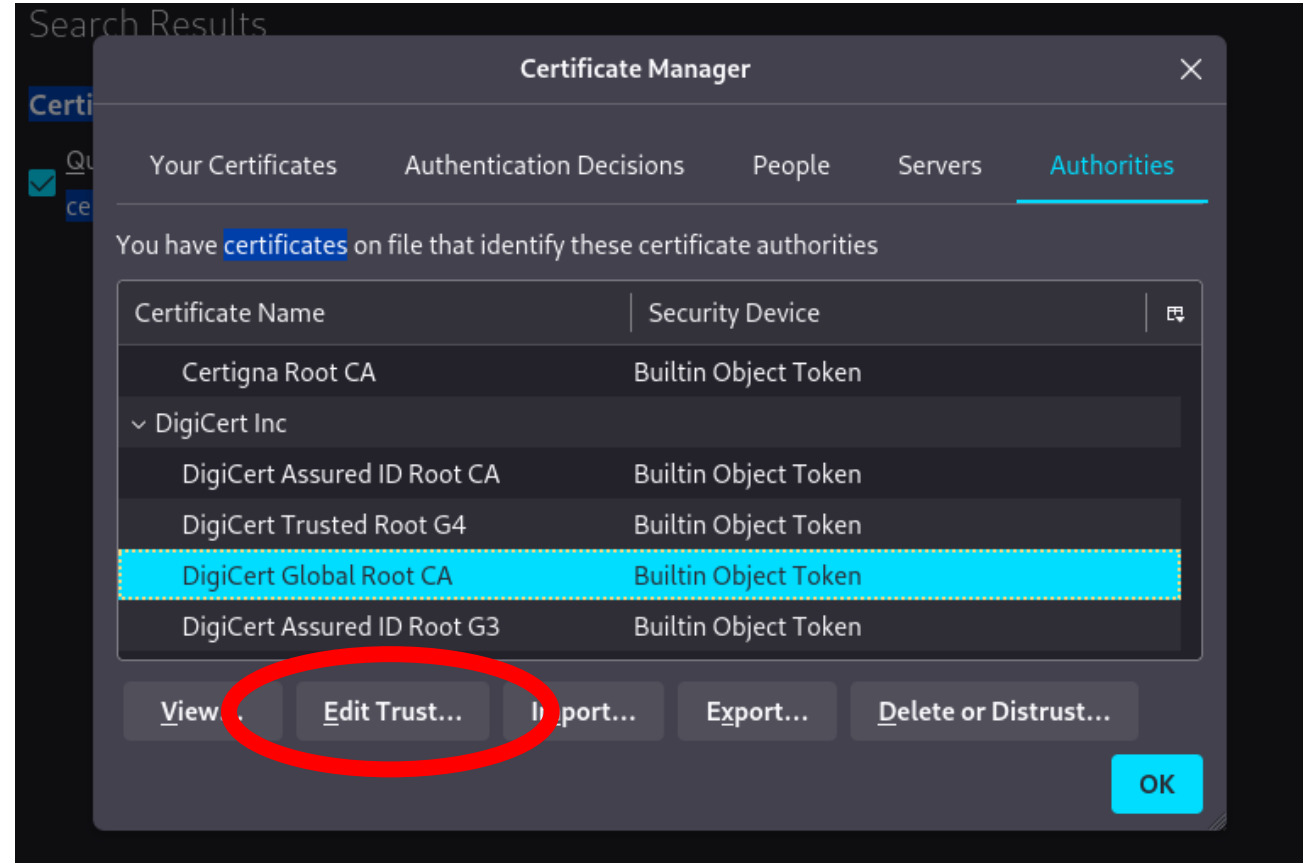






## Root Certificate Authorities

- Under the 'Authorities' tab, find the "DigiCert Global Root CA" certificate that was ultimately used to verify the authenticity of IBM's certificate. Click "Edit Trust."





## Question 6:

What can this DigiCert root certificate be used for?

Edit trust settings:

- ☒ This certificate can identify websites.
- ☒ This certificate can identify mail users.



# Digital Certificates and Signature



- Now find any of the root certificates for **Symantec Corporation**. Select one and click “Edit Trust.”

## Question 7:

What can this Symantec root certificate be used for? What can it not be used for?

- Examine other certs in the list to see what a “normal” cert in Firefox’s store can do.



## Why is the trust for Symantec's cert anomalous?

Dating back to 2015, **Symantec** had been called out for some minor **mis-issuance** issues. In 2016, upon investigating further, Google found other evidence of **mis-issuance** and that Symantec was practicing lax oversight over the regional authorities it was outsourcing validation to in various regions.



## Symantec vs Google

Symantec and Google's headquarters are across the street from each other, which made for some awkward bus rides to work...

- Symantec, rightfully, points out that there were a total of 33 mis-issued test certificates and that no real-world harm actually occurred as a result of it.
- Google and the other browsers argued that Symantec's mis-issuance problems were systemic and by extension, they could no longer trust the certificates Symantec CA brands were issuing.



## **Google and Symantec reached an agreement:**

- over the Summer of 2017 that Symantec would shift issuance of certificates to a managed CA while it rebuilt its PKI.
- In the meantime, Symantec's roots would be distrusted, which in turn distrusts all of Symantec's certificates.
- That was what facilitated Symantec selling its CA to DigiCert for nearly a billion dollars and a 30% share.



# OpenSSL Demo



In this lab session we will implement all the steps of a secure communication protocol between **Alice** and **Bob**, with the support of the Certification Authority **kaust\_ca**, using **OpenSSL** terminal commands.

**Goal:** Alice needs to send a file to Bob, over an insecure communication channel, ensuring **confidentiality** and **integrity**.

1. Alice and Bob get a certificate from kaust\_ca
2. Alice and Bob exchange their certificates over an insecure communication channel.
3. After verifying Bob's certificate using kaust\_ca certificates, Alice encrypt the file and generates a signature. Finally, Alice send to Bob the encrypted file and the signature.
4. Bob opens the file and verifies the signature



# OpenSSL Demo



In your Kali Linux VM, create one folder for this demo:

```
mkdir openssl_demo
```

Go to the demo directory with **cd openssl\_demo**

Then, create inside this folder one more folder for **alice**, one for **bob**, and one for **kaust\_ca**





## 1.a - Alice generates a private key



```
$ openssl genpkey -algorithm RSA -pkeyopt  
rsa_keygen_bits:2048 -pkeyopt rsa_keygen_pubexp:3 -out  
alice/privkey-A.pem
```

- **genpkey** → generate a private key
- **algorithm RSA** → use the RSA algorithm (can also take “EC” for elliptic-curve)
- **rsa\_keygen\_bits:2048** → sets the size of the key to 2048 bits (the default is 1024)
- **rsa\_keygen\_pubexp:3** → sets the public exponent  $e$  to 3 (default is 65, 537)
- **out privkey-A.pem** → outputs to the file privkey-A.pem



## 1.b - Alice generates a public key



```
$ openssl pkey -in alice/privkey-A.pem -pubout -out  
alice/pubkey-A.pem
```

- pkey → processes public or private keys
- -in privkey-A.pem → read the key from filename privkey-A.pem
- -pubout → output a public key (by default, a private key is output)

Repeat Steps **1.a** and **1.b** for **bob** and the **CA**



## Step 1.c - Alice generates a certificate signing request

```
$ openssl req -new -key alice/privkey-A.pem -out  
alice/A-req.csr
```

- req → creates and processes signing requests
- -new → generates a new certificate request, will prompt Alice for some information
- -key privkey-A.pem → signs the request with Alice's private key
- **Important: you must fill (at least) the “common name” field! It can be a random string, but it must be unique (the Alice's one is different from Bob's one)**
- **Repeat this step for bob**



## Aside: generating a self-signed certificate for the CA



```
$ openssl req -x509 -new -nodes -key  
kaust_ca/privkey-CA.pem -sha256 -days 1024  
-out kaust_ca/root.crt
```

Then, send the certificate to Alice and Bob:

```
$ cp kaust_ca/root.crt alice  
$ cp kaust_ca/root.crt bob
```



## 1.d - CA generates and signs a certificate for Alice



```
$ openssl x509 -req -in alice/A-req.csr -CA kaust_ca/root.crt -CAkey kaust_ca/privkey-CA.pem -CAcreateserial -out alice/A.crt -days 500 -sha256
```

- **x509** → an x509 certificate utility (displays, converts, edits and signs x509 certificates)
- **-req** → a certificate request is taken as input (default is a certificate)
- **-CA root.crt** → specifies the CA certificate to be used as the issuer of Alice's certificate
- **-CAkey rootkey.pem** → specifies the private key used in signing (rootkey.pem)
- **-CAcreateserial** → creates a serial number file which contains a counter for how many certificates were signed by this CA



## 1.d - CA generates and signs a certificate for Bob



```
$ openssl x509 -req -in bob/B-req.csr -CA kaust_ca/root.crt -  
CAkey kaust_ca/privkey-CA.pem -CAcreateserial -out bob/B.crt  
-days 500 -sha256
```

Then, Bob sends its certificate to Alice:

```
$ cp bob/B.crt alice/
```



## Step 2.a - Alice verifies Bob's public certificate



Repeat the previous steps (1.d) for bob, then send Bob's certificate to Alice  
Alice verifies Bob's public certificate:

```
$ openssl verify -CAfile alice/root.crt alice/B.crt
```

If you get an OK, you know the certificate can be trusted!



## Step 2.b - Alice extracts Bob's public key and Encrypt



```
$ openssl x509 -pubkey -in alice/B.crt -noout > alice/pubkey-B.pem
```

Alice creates a file and encrypts it using Bob's public key

```
$ echo "This is a private file" > alice/myfile.txt
```

```
$ openssl pkeyutl -encrypt -in alice/myfile.txt -pubin -inkey  
alice/pubkey-B.pem -out alice/myfile.enc
```





## Step 2.c – Additional Security



Alice hashes myfile.txt and encrypts it using her private key

```
$ openssl dgst -sha1 -sign alice/privkey-A.pem -out  
alice/signature.bin alice/myfile.txt
```

Then, Alice sends both the encrypted file and the hash to Bob

```
$ cp alice/myfile.enc alice/signature.bin bob
```



## Step 3 – File Decryption



Bob decrypts the received file using his private key

```
$ openssl pkeyutl -decrypt -in bob/myfile.enc -inkey  
bob/privkey-B.pem -out bob/myfile.txt
```

Then, - Bob gets and verifies Alice's certificate and extracts her public key

**Can you do that?**



## Step 3.a – File Decryption



Finally, Bob verifies whether the message is coming from Alice

```
$ openssl dgst -sha1 -verify bob/pubkey-A.pem -  
signature bob/signature.bin bob/myfile.txt
```

If you get an OK, the communication protocol is successful !!

# Do you have any questions?



[rc3.kaust.edu.sa](https://rc3.kaust.edu.sa)



Follow us @rc3kaust

